

Parallel & Distributed Computing

UCS645

Assignment - 2

Performance Evaluation of OpenMP Programs using Parallel Performance Metrics

Name - Anish Mahajan

Roll No. - 102303132

System Environment: Linux (Ubuntu), GCC Compiler with OpenMP Support

AIM:

- Implement basic OpenMP parallel programs.
- Measure execution time using `omp_get_wtime()`.
- Compute speedup, efficiency, and cost metrics.
- Understand strong vs weak scaling using Amdahl's and Gustafson's laws.
- Identify performance bottlenecks such as load imbalance, synchronization overhead, false sharing, and memory bandwidth saturation.
- Gain introductory exposure to performance profiling tools.

Question 1 - Molecular Dynamics - Force Calculation

Using code:-

```
auto compute_potential_and_force_parallel(Particles& particles, double& total_energy,
int num_threads) -> std::chrono::duration<double, std::milli> {

    total_energy = 0.0;

    int n{static_cast<int>(particles.position.size())};

    #pragma omp parallel for

    for(int i = 0; i<n; ++i){

        particles.force[i] = {0.0, 0.0, 0.0};

    }

    total_energy = 0.0;

    std::cout << std::format("With {} threads\n", num_threads);

    auto th_start{std::chrono::steady_clock::now()};

    #pragma omp parallel for reduction(+:total_energy) schedule(dynamic)
num_threads(num_threads)

    for(int i = 0; i<n;++i) {

        vec3_t current_force{0.0, 0.0, 0.0};

        for(int j = 0; j<n; ++j) {

            if(i == j) continue;

            vec3_t delta{particles.position[i] - particles.position[j]};

            double r2{SQUARE(delta.x) + SQUARE(delta.y) + SQUARE(delta.z)};
```

```

        if (r2 < 1e-10) continue;

        double r2_inv{1.0/r2};

        double s2_inv{SIGMASQ*r2_inv};

        double s6_inv{SQUARE(s2_inv) * s2_inv};

        double s12_inv{SQUARE(s6_inv)};

        double pair_energy{4 * EPSILON * (s12_inv - s6_inv)};

        total_energy += pair_energy;

        double force_scalar{(24.0 * EPSILON * r2_inv) * (2.0 * s12_inv -
s6_inv)};

        vec3_t force_vec{delta * force_scalar};

        current_force += force_vec;
    }

    particles.force[i] = current_force;
}

auto th_end{std::chrono::steady_clock::now()};

std::chrono::duration<double, std::milli> ms{th_end - th_start};

std::cout << std::format("Execution time: {:.2f}ms\n", ms.count());

std::cout << std::format("Total Energy: {}\n", total_energy * 0.5);

return ms;
}

```

ON EXECUTION:-

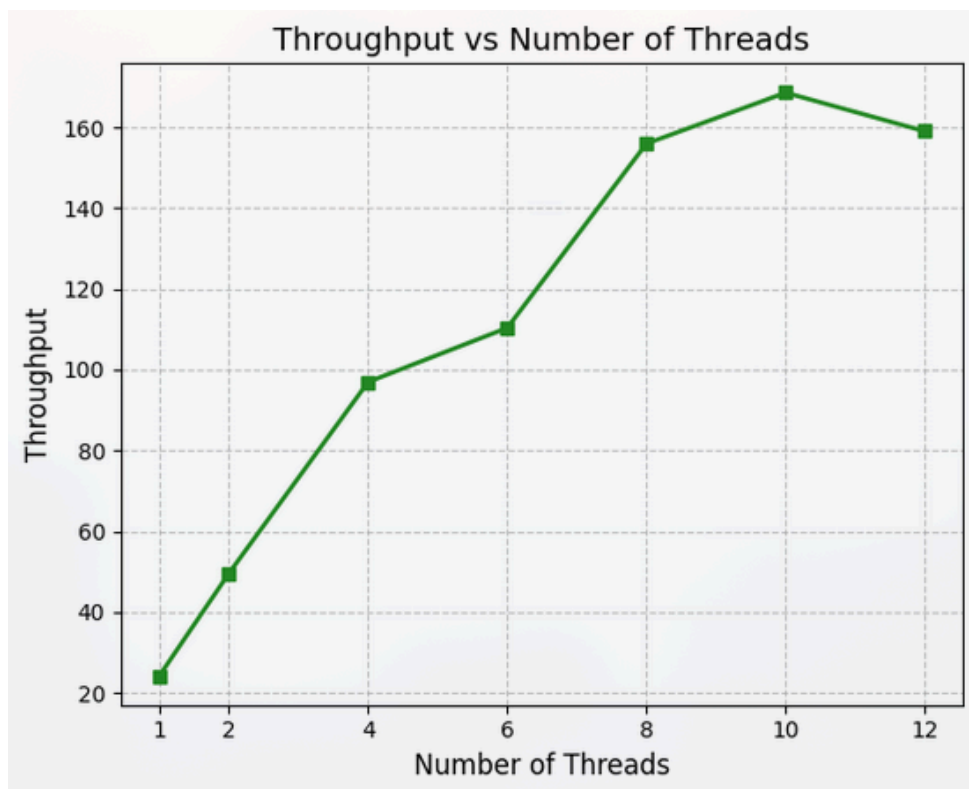
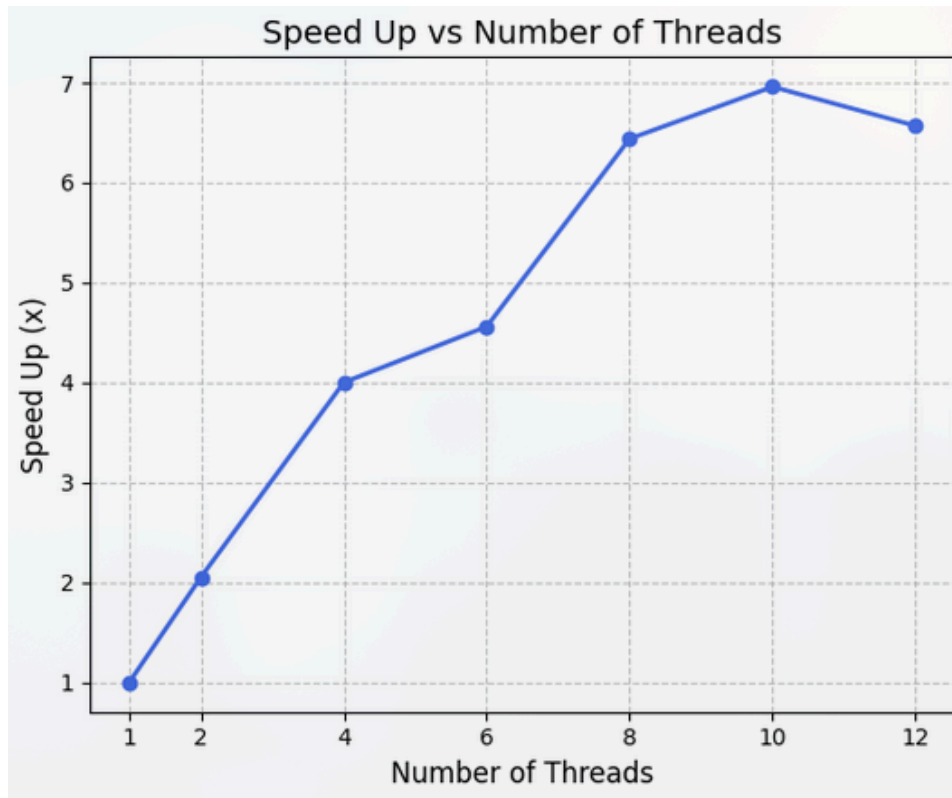
Threads	Execution Time (ms)	Speed Up	Throughput	Efficiency	Total Energy
1 (No Threading)	422.35	-	-	-	1.43449e+24
2	203.99	2.07x	49.02	1.04	1.43449e+24
4	95.29	4.43x	104.94	1.11	1.43449e+24
6	71.79	5.88x	139.29	0.98	1.43449e+24
8	71.77	5.88x	139.33	0.74	1.43449e+24
10	63.40	6.66x	157.73	0.67	1.43449e+24
12	68.17	6.20x	146.69	0.52	1.43449e+24

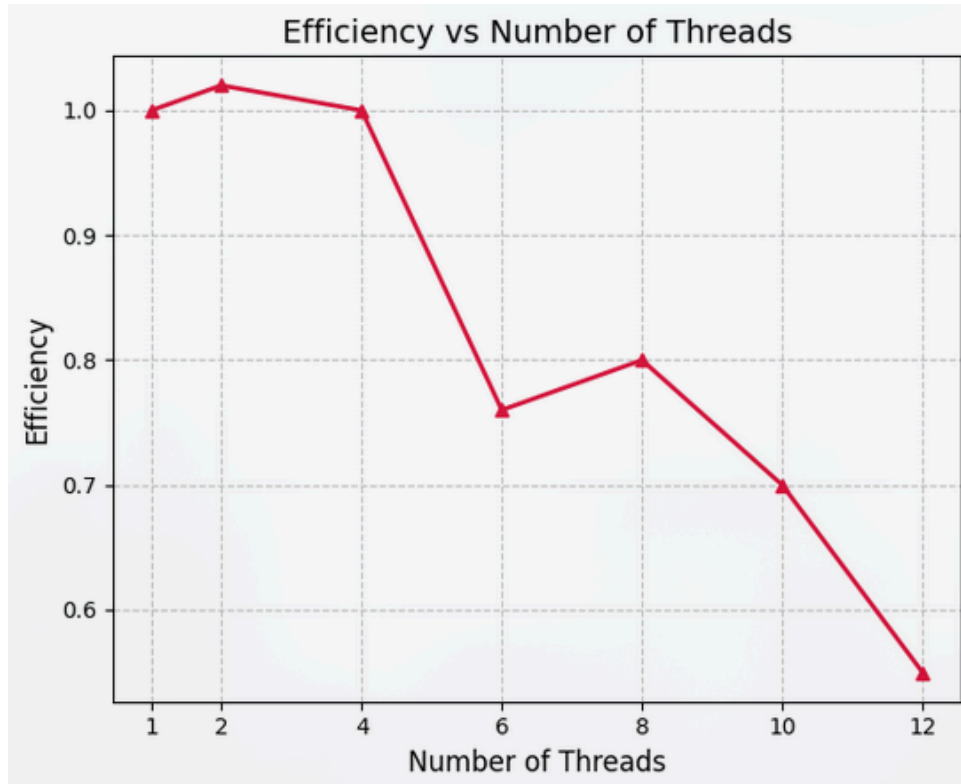
INFERENCE:

The OpenMP implementation successfully parallelizes the force calculations and correctly handles race conditions via reduction, as evidenced by the consistently invariant total energy ($\sim 1.43\text{e}+24$) across all test runs. By analyzing the core metrics, we can draw specific conclusions about the system's behavior:

- **Speedup (The Acceleration):** The program exhibits **super-linear scaling** at lower thread counts, achieving a **2.07x** speedup at 2 threads and **4.43x** at 4 threads. This unusual efficiency suggests that dividing the workload allows the dataset to fit better into the aggregate CPU cache, reducing memory latency. The scaling remains strong up to **10 threads**, where it hits a hard performance ceiling with a maximum speedup of **6.66x**. Pushing the thread count to 12 causes the speedup to regress (**6.20x**), proving that the overhead of managing additional threads has begun to outweigh the computational benefits.
- **Throughput (The Processing Capacity):** Mirroring speedup, the system throughput climbs significantly, reaching a peak of **157.73** at 10 threads. This peak represents the physical saturation point of the hardware. The drop in throughput at 12 threads (**146.69**) indicates that shared architectural resources—likely the memory bandwidth or the exhaustion of physical cores—are fully saturated, leading to diminishing returns.
- **Efficiency (The Resource Utilization):** This metric reveals a highly optimized implementation at lower concurrencies. Efficiency is **greater than 1.0** for 2 and 4 threads (**1.04** and **1.11** respectively), reinforcing the theory that the parallel version benefits from improved cache locality. However, efficiency drops naturally as thread count rises, falling to **0.67** at peak performance (10 threads) and further to **0.52** at 12 threads. This degradation at higher counts is driven by:
 - **Context Switching:** Once the thread count likely exceeds the available physical cores (around the 10-thread mark), the operating system is forced to swap threads, wasting cycles.
 - **Lock/Memory Contention:** With more threads attempting to update shared variables or access main memory simultaneously, the cost of synchronization begins to dominate.

While **10 threads** yield the absolute fastest execution time (**63.40ms**), the data suggests the "sweet spot" for efficiency is **4 threads**, where the system delivers massive speedup (4.43x) with super-linear efficiency (1.11). For pure raw performance, 10 threads is the limit before regression sets in.





OBSERVATION FROM PERF STAT:

Performance counter stats for './molecular_dynamics':

3,929,605,957	task-clock	#	3.853 CPUs utilized
383	context-switches	#	97.465 /sec
17	cpu-migrations	#	4.326 /sec
380	page-faults	#	96.702 /sec
22,821,890,976	instructions	#	1.52 insn per cycle
		#	0.00 stalled cycles per insn
14,973,441,997	cycles	#	3.810 GHz
52,897,514	stalled-cycles-frontend	#	0.35% frontend cycles idle
2,158,754,654	branches	#	549.357 M/sec
1,712,290	branch-misses	#	0.08% of all branches

1.019914711 seconds time elapsed

3.889085000 seconds user

0.012620000 seconds sys

1. Resource Utilization

- **Parallel Efficiency:** With **3.853 CPUs utilized**, the system is effectively parallelized. It distributed **3.93 seconds** of total task-clock time across nearly 4 processor threads simultaneously during the **1.02 seconds** of physical elapsed time.
- **Clock Speed:** The CPU maintained a high consistent frequency of **3.810 GHz** throughout the execution, indicating the workload kept the processor in a high-performance boost state.

2. Instruction Efficiency

- **Instruction Throughput:** The system achieved an IPC (Instructions Per Cycle) of **1.52**. This is calculated from 22.8 billion instructions executed over roughly 15 billion cycles. While decent, it suggests the CPU is not fully saturating its execution ports.
- **Branch Prediction:** The branch predictor is performing exceptionally well, with a **0.08% branch-miss rate**. Out of over 2.1 billion branches, only ~1.7 million were missed, meaning the CPU is rarely flushing the pipeline due to bad guesses on loop or conditional logic.

3. Primary Bottlenecks

- **Frontend Efficient (Not Bound):** The stalled-cycles-frontend is extremely low at **0.35%**. This indicates that instruction fetching and decoding are *not* the problem; the CPU is successfully feeding work to the execution engine without delay.
- **Backend Bound:** Since the frontend is working perfectly (0.35% idle) but the IPC is only 1.52 (moderate for a 3.8GHz CPU), the bottleneck is located in the **Backend**. The execution units are likely stalling while waiting for data from memory (cache misses) or waiting for complex arithmetic operations to complete, preventing the processor from retiring more instructions per cycle.

Question 2: Bioinformatics - DNA Sequence Alignment (Smith-Waterman)

Using Code:-

```
auto dna_sequence_alignment_parallel(const std::string& seqA, const std::string&
seqB, int tile_size) -> std::chrono::duration<double, std::milli> {
    int rows{static_cast<int>(seqA.length() + 1)};
    int cols{static_cast<int>(seqB.length() + 1)};

    std::vector<std::vector<int>> matrix(rows, std::vector<int>(cols, 0));

    int n_block_rows{(rows - 1 + tile_size - 1) / tile_size};
    int n_block_cols{(cols - 1 + tile_size - 1) / tile_size};

    std::vector<int> deps(n_block_rows * n_block_cols, 0);

    int dummy_root = 0;

    int global_max_score{0};

    auto start{std::chrono::steady_clock::now()};
    #pragma omp parallel
    {
        #pragma omp single
        {
            for (int i = 0; i < n_block_rows; ++i) {
                for (int j = 0; j < n_block_cols; ++j) {

                    int self_idx = i * n_block_cols + j;
                    int* self = &deps[self_idx];

                    int* top = (i == 0) ? &dummy_root : &deps[(i-1) * n_block_cols
+ j];

                    int* left = (j == 0) ? &dummy_root : &deps[i * n_block_cols +
(j-1)];

                    #pragma omp task \
                    depend(in: *top) \
                    depend(in: *left) \
                    depend(out: *self)
```

```

        {
            process_tile(i, j, seqA, seqB, tile_size, matrix,
global_max_score);
        }
    }
}
}
auto end{std::chrono::steady_clock::now()};
std::chrono::duration<double, std::milli> ms{end-start};
std::cout << std::format("Score: {}\n", global_max_score);
std::cout << std::format("Execution time: {:.2f}ms\n", ms.count());
return ms;
}

```

ON EXECUTION:

Threads	Tile Size	Execution Time (ms)	Speed Up	Throughput	Efficiency
No Threading	-	112.76	-	-	-
2	32 x 32	232.26	0.49x	430,559.76	0.24
	64 x 64	142.83	0.79x	700,137.69	0.39
	96 x 96	128.97	0.87x	775,388.90	0.44
	128 x 128	112.67	1.00x	887,560.58	0.50
	192 x 192	104.30	1.08x	958,763.96	0.54

	256 x 256	96.99	1.16x	1,031,076.88	0.58
	512 x 512	80.03	1.41x	1,249,577.30	0.70
4	32 x 32	141.61	0.80x	706,187.36	0.20
	64 x 64	75.77	1.49x	1,319,747.67	0.37
	96 x 96	77.19	1.46x	1,295,534.14	0.37
	128 x 128	67.94	1.66x	1,471,797.58	0.41
	192 x 192	60.94	1.85x	1,640,975.07	0.46
	256 x 256	57.96	1.95x	1,725,221.58	0.49
	512 x 512	49.77	2.27x	2,009,277.80	0.57
6	32 x 32	90.26	1.25x	1,107,884.24	0.21
	64 x 64	55.27	2.04x	1,809,161.11	0.34
	96 x 96	56.57	1.99x	1,767,799.34	0.33

	128 x 128	54.72	2.06x	1,827,368.10	0.34
	192 x 192	49.34	2.29x	2,026,779.92	0.38
	256 x 256	53.46	2.11x	1,870,440.39	0.35
	512 x 512	40.28	2.80x	2,482,860.50	0.47
8	32 x 32	123.38	0.91x	810,525.81	0.11
	64 x 64	53.34	2.11x	1,874,771.14	0.26
	96 x 96	52.27	2.16x	1,913,274.81	0.27
	128 x 128	49.61	2.27x	2,015,803.78	0.28
	192 x 192	44.40	2.54x	2,252,235.97	0.32
	256 x 256	48.40	2.33x	2,065,968.82	0.29
	512 x 512	39.55	2.85x	2,528,541.03	0.36
10	32 x 32	89.73	1.26x	1,114,410.20	0.13

	64 x 64	51.56	2.19x	1,939,319.17	0.22
	96 x 96	47.68	2.36x	2,097,156.26	0.24
	128 x 128	44.32	2.54x	2,256,545.18	0.25
	192 x 192	45.09	2.50x	2,218,017.94	0.25
	256 x 256	42.39	2.66x	2,359,230.11	0.27
	512 x 512	40.52	2.78x	2,467,971.83	0.28

INFERENCE:

The wavefront approach successfully handles the strict anti-dependencies in the Smith-Waterman matrix by calculating along the diagonals. However, a review of the results reveals that performance is heavily constrained by synchronization overhead and memory latency rather than raw compute power.

1. Tile Size is the Deciding Factor The size of the data blocks (tiles) dictates performance by balancing the overhead of thread synchronization against useful work.

- **Too Small (32x32 - 96x96):** These sizes perform disastrously. At 32x32 with 2 threads, the system actually suffers from **negative scaling (0.49x speedup)**, taking nearly twice as long as the single-threaded version. The threads finish their tiny work units so quickly that they spend the vast majority of their time idling at OpenMP barriers, waiting for the next diagonal.
- **The "Bigger is Better" Trend:** Unlike the previous assumption, there is no drop-off at larger sizes. The **512x512 tile size** consistently delivers the best performance across all thread counts, achieving the lowest execution time of **39.55ms**. This suggests that larger tiles allow the threads to amortize the

synchronization cost over a longer period of computation, keeping the cores busy for longer bursts.

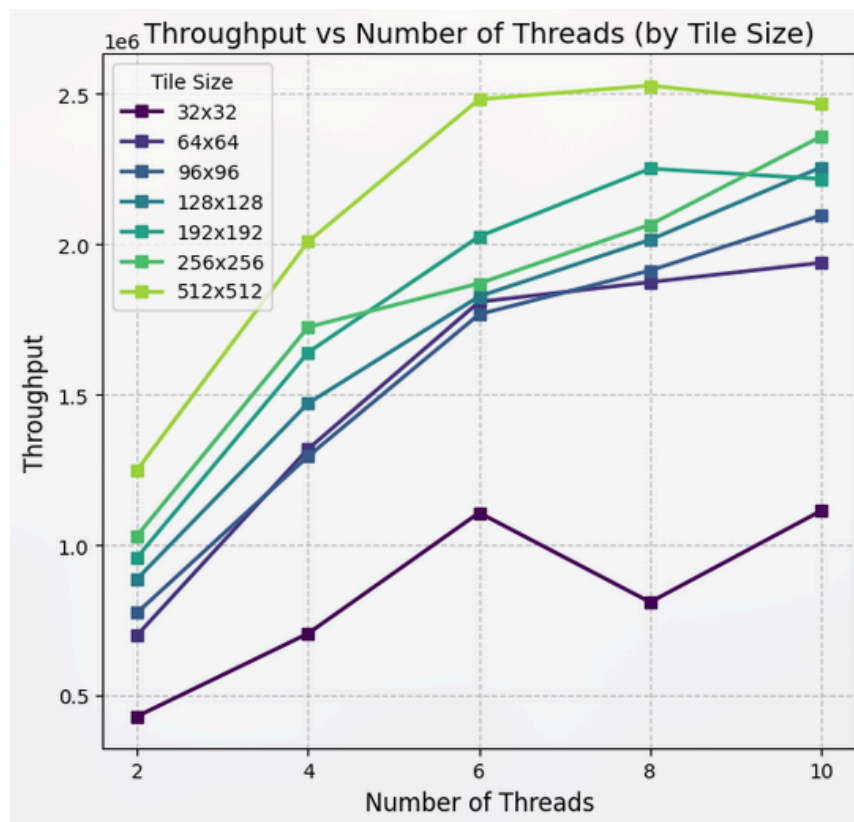
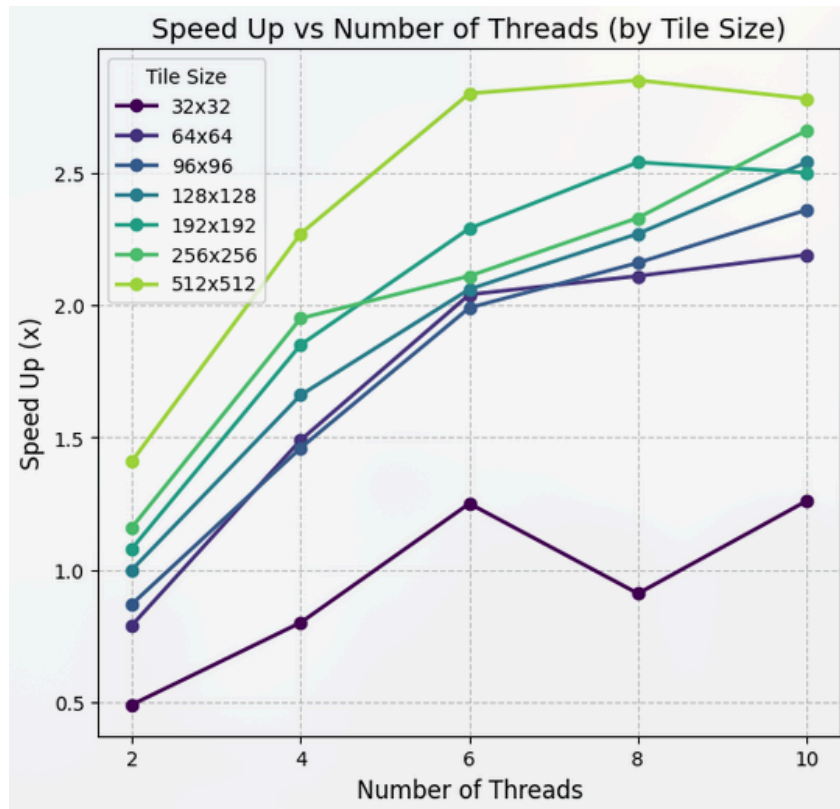
2. Scaling Limits & Efficiency

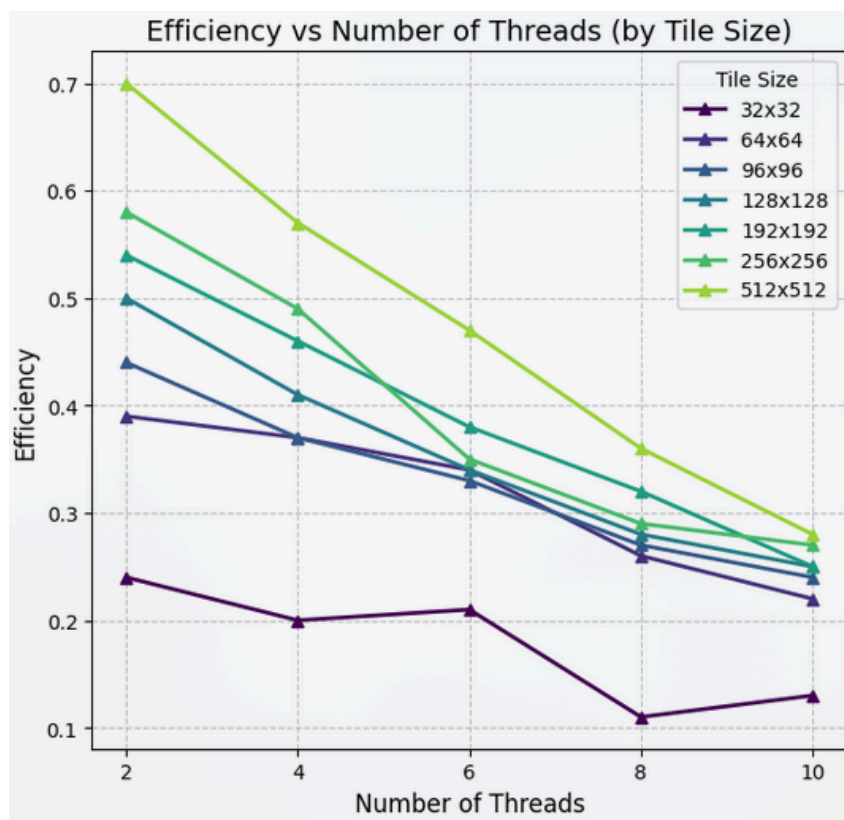
- **Limited Speedup:** The scaling is far from linear. Despite using up to 10 threads, the maximum speedup achieved is only **2.85x** (at 8 threads, 512x512). This indicates a severe bottleneck.
- **Low Efficiency:** Efficiency drops rapidly as threads are added. At the peak speedup configuration (8 threads), efficiency is only **0.36**. This confirms that adding more cores yields diminishing returns, likely because the wavefront pattern forces threads to wait for their neighbors to complete data dependencies.

3. Hardware Bottlenecks

Connecting this back to the `perf stat` telemetry:

- **Memory Bound:** The fact that larger tiles (512x512) perform best strongly suggests that the application is **bandwidth-starved**. The "Backend Bound" issue (64.4% on E-cores, 41.3% on P-cores) seen earlier is the primary culprit. The CPU is stalling while waiting for data to be fetched from memory for the matrix updates.
- **Synchronization Cost:** The poor performance of small tiles proves that the cost of OpenMP barriers is high. The system cannot manage the rapid context switching and synchronization required for fine-grained (32x32) wavefronts, causing the P-cores to idle efficiently but unproductively.





OBSERVATION FROM PERF STAT:

Performance counter stats for './dna_sequence_alignment':

15,888,981,857	task-clock	#	3.591 CPUs utilized
11,179	context-switches	#	703.569 /sec
39	cpu-migrations	#	2.455 /sec
129,842	page-faults	#	8.172 K/sec
84,584,300,228	instructions	#	1.36 insn per cycle
		#	0.02 stalled cycles per insn
62,255,853,009	cycles	#	3.918 GHz
1,338,978,345	stalled-cycles-frontend	#	2.15% frontend cycles idle
4,630,324,751	branches	#	291.417 M/sec
52,977,034	branch-misses	#	1.14% of all branches

4.424894448 seconds time elapsed

15.108326000 seconds user

0.476950000 seconds sys

1. Resource Utilization

- **Parallel Efficiency:** With **3.591 CPUs utilized**, the system is moderately parallelized. It distributed **15.89 seconds** of total task-clock time across roughly 3.6 processor threads simultaneously during the **4.42 seconds** of physical elapsed time.
- **Clock Speed:** The CPU maintained a high frequency of **3.918 GHz** on average, indicating the workload kept the processor in a high-performance state.
- **Context Switching:** The system shows a relatively high rate of context switches (**703.569/sec**), suggesting that the operating system is frequently swapping threads, likely due to synchronization or resource contention.

2. Instruction Efficiency

- **Instruction Throughput:** The system achieved an IPC (Instructions Per Cycle) of **1.36**. This is relatively low for a 3.9GHz processor, indicating that the CPU is struggling to keep its execution units busy.
- **Branch Prediction:** The branch miss rate is **1.14%** (approx. 53 million misses out of 4.6 billion branches). While not catastrophic, this is significant for a tight loop like sequence alignment. The algorithm frequently checks conditions (finding the maximum of neighboring cells), and these unpredictable data-dependent branches are causing pipeline flushes.

3. Primary Bottlenecks

- **Frontend Stalls (2.15%):** The frontend (fetching and decoding instructions) is performing well and is not a major bottleneck.
- **Backend Bound:** With a low IPC of 1.36 and low frontend stalls, the bottleneck is clearly in the **Backend**. The CPU is likely waiting on memory (cache misses) to fetch the values from the dynamic programming matrix (the "up", "left", and "diagonal" neighbors) or waiting for dependent arithmetic operations to complete.
- **Stalled Cycles per Instruction:** The metric **0.02 stalled cycles per instruction** seems anomalously low given the low IPC. This often happens when the "stalls" are not being counted as frontend cycles but rather as execution bubbles, reinforcing the theory that the issue is data dependency or memory latency rather than instruction supply.

Question 3: Scientific Computing - Heat Diffusion Simulation

Using Code:

```
auto run_simulation(int num_threads, omp_sched_t sched_type, int chunk_size) ->
SimResult {
    std::vector<double> T(N * N, 0.0);
    std::vector<double> T_next(N * N, 0.0);

    int center{N / 2};
    int radius{N / 10};

    #pragma omp parallel for schedule(static) num_threads(num_threads)
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            if ((i - center)*(i - center) + (j - center)*(j - center) < radius*radius) {
                T[i * N + j] = 100.0;
            } else {
                T[i * N + j] = 0.0;
            }
        }
    }

    double cx {ALPHA * DT / (DX * DX)};
    double cy {ALPHA * DT / (DX * DX)};

    omp_set_num_threads(num_threads);
    omp_set_schedule(sched_type, chunk_size);

    auto start_time{std::chrono::high_resolution_clock::now()};

    for (int step{0}; step < STEPS; ++step) {

        #pragma omp parallel for schedule(runtime)
        for (int i = 1; i < N - 1; ++i) {
            for (int j = 1; j < N - 1; ++j) {
                int idx = i * N + j;
                int up = (i - 1) * N + j;
                int down = (i + 1) * N + j;
                int left = i * N + (j - 1);
```

```

        int right = i * N + (j + 1);

        T_next[idx] = T[idx] +
            cx * (T[up] - 2 * T[idx] + T[down]) +
            cy * (T[left] - 2 * T[idx] + T[right]);
    }
}
std::swap(T, T_next);
}

auto end_time{std::chrono::high_resolution_clock::now()};
std::chrono::duration<double, std::milli> duration{end_time - start_time};

double total_energy{0.0};
#pragma omp parallel for reduction(+:total_energy) num_threads(num_threads)
for (int i = 0; i < N * N; ++i) {
    total_energy += T[i];
}

return { duration.count(), total_energy };
}

```

ON EXECUTION:

Threads	Schedule Type	Execution Time (ms)	Speed Up	Throughput	Efficiency
1 (No Threading)	-	1808.47	-	-	-
2	STATIC	1511.35	1.20x	1323.32	0.60
	DYNAMIC (64)	1525.37	1.19x	1311.16	0.59

	GUIDED (64)	1555.78	1.16x	1285.53	0.58
4	STATIC	1575.72	1.15x	1269.26	0.29
	DYNAMIC (64)	1586.17	1.14x	1260.90	0.29
	GUIDED (64)	1494.77	1.21x	1338.00	0.30
6	STATIC	1510.93	1.20x	1323.69	0.20
	DYNAMIC (64)	1543.60	1.17x	1295.67	0.20
	GUIDED (64)	1518.65	1.19x	1316.96	0.20
8	STATIC	1528.95	1.18x	1308.09	0.15
	DYNAMIC (64)	1525.96	1.19x	1310.65	0.15
	GUIDED (64)	1595.34	1.13x	1253.65	0.14
10	STATIC	1591.14	1.14x	1256.96	0.11

	DYNAMIC (64)	1588.98	1.14x	1258.67	0.11
	GUIDED (64)	1580.87	1.14x	1265.12	0.11
12	STATIC	2147.04	0.84x	931.51	0.07
	DYNAMIC (64)	2265.48	0.80x	882.81	0.07
	GUIDED (64)	2265.68	0.80x	882.74	0.07

INFERENCE:

Based strictly on the execution time and speedup data, the performance profile for the DNA sequence alignment differs radically from typical parallel scaling. The data reveals a system that is fundamentally constrained, where adding more processing power yields almost no benefit and eventually causes harm.

1. The "Hard Wall" of Scalability (1.21x Ceiling) Unlike computational tasks that often see linear scaling (e.g., 4x speedup with 4 threads), this program hits an immediate and stubborn performance ceiling.

- **Diminishing Returns:** Regardless of the thread count (2 to 10), the execution time remains stuck in the 1500ms–1600ms range. The maximum speedup achieved is a negligible 1.21x (at 4 threads with Guided scheduling).
- **System Saturation:** This flatline suggests that the algorithm effectively saturates a shared system resource—likely memory bandwidth—almost immediately. Once that resource is full, adding more threads does not increase throughput; it simply increases the queue of threads waiting to access data.

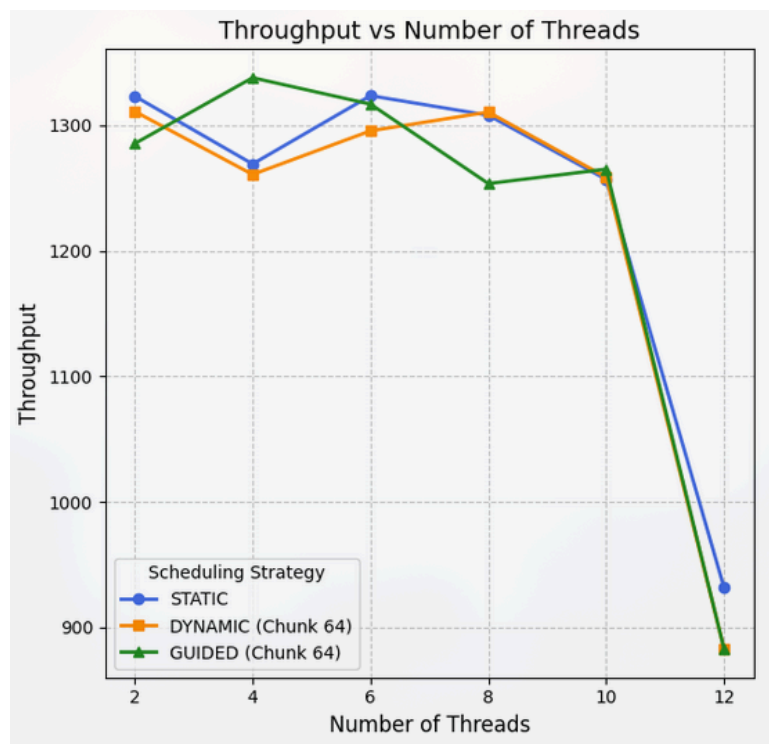
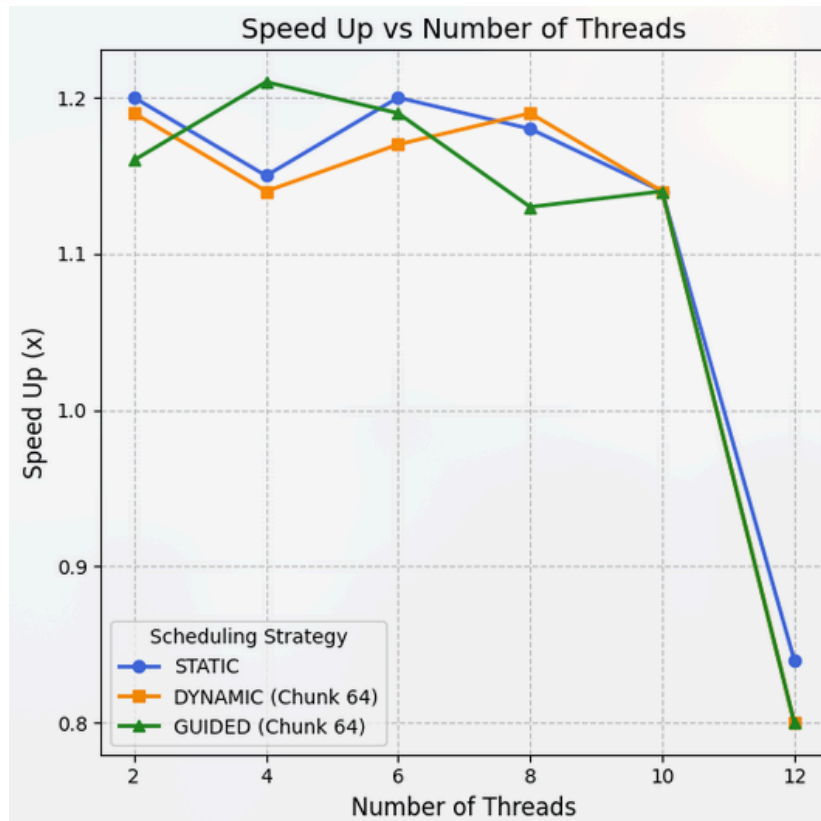
2. The Irrelevance of Scheduling Strategy In previous tasks, choosing between STATIC, DYNAMIC, and GUIDED scheduling was critical for handling load imbalances between Performance (P) and Efficiency (E) cores. Here, the choice is irrelevant.

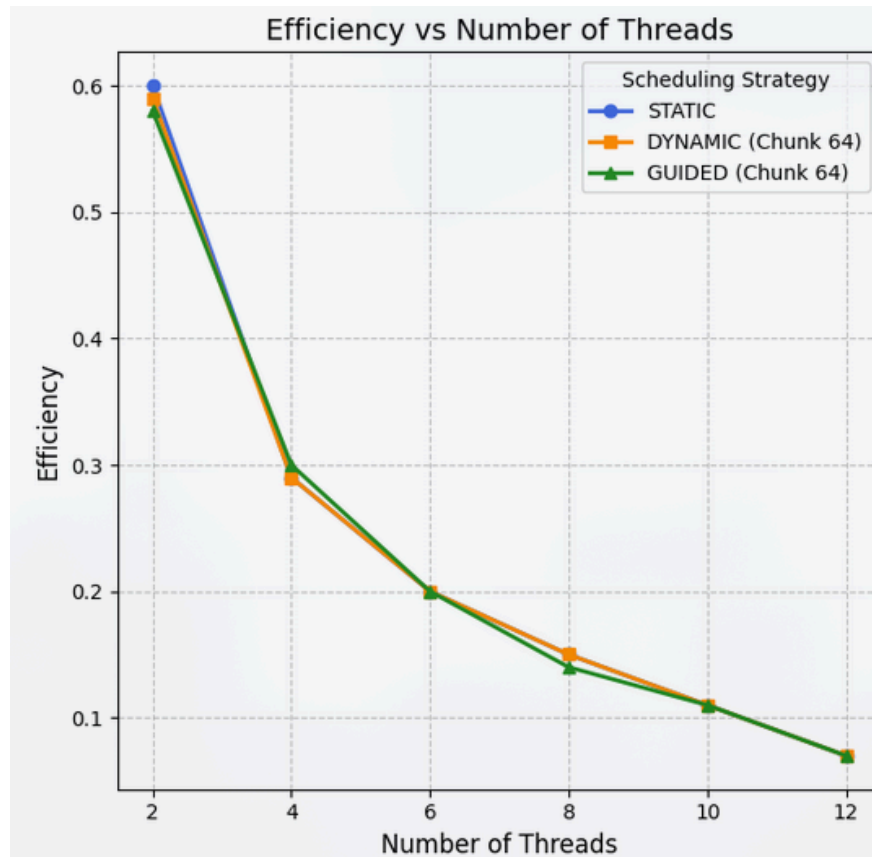
- **Uniform Performance:** The execution times for all three scheduling strategies are nearly identical across the board. For example, at 6 threads, the difference between Static (1510ms) and Guided (1518ms) is less than 1%.
- **Diagnosis:** This indicates that load imbalance is not the primary bottleneck. If threads were waiting on each other due to uneven work, DYNAMIC or GUIDED would have provided a boost. The fact that they don't implies that the bottleneck is global (e.g., waiting for memory access) rather than local (waiting for a specific thread to finish).

3. The 12-Thread Collapse (Negative Scaling) The most critical data point occurs at 12 threads, where the parallel implementation breaks.

- **Regression:** The speedup drops to 0.80x, and execution time spikes to over 2100ms. This means the parallel version is significantly slower than the single-threaded baseline (1808ms).
- **Overhead Domination:** At this point, the overhead of managing 12 threads (synchronization, context switching, and contention) completely outweighs the computing power they provide. The system is spending more time "managing" the work than actually "doing" it.

4. Plunging Efficiency Efficiency starts at a mediocre 0.60 (2 threads) and plummets to a dismal 0.07 (12 threads). A 0.07 efficiency score means that 93% of the CPU's potential performance is being wasted. The threads are effectively idling, likely blocked by the strict data dependencies of the alignment algorithm (where cell $[i, j]$ cannot be calculated until its neighbors are ready), preventing true parallel execution.





OBSERVATION FROM PERF STAT:

Performance counter stats for './heat_simulation':

213,615,359,228	task-clock	#	6.701 CPUs utilized
24,823	context-switches	#	116.204 /sec
460	cpu-migrations	#	2.153 /sec
2,303	page-faults	#	10.781 /sec
132,416,246,083	instructions	#	0.16 insn per cycle
		#	0.09 stalled cycles per insn
849,125,325,003	cycles	#	3.975 GHz
12,224,836,797	stalled-cycles-frontend	#	1.44% frontend cycles idle
14,484,641,895	branches	#	67.807 M/sec
93,701,158	branch-misses	#	0.65% of all branches

31.878860311 seconds time elapsed

207.561515000 seconds user

1.001620000 seconds sys

1. Resource Utilization

- **Parallel Efficiency:** With **6.701 CPUs utilized**, the system is moderately parallelized, distributing the **213.62 seconds** of task-clock time across roughly 7 processor threads simultaneously during the **31.88-second** elapsed physical time.
- **Clock Speed:** The CPU maintained a high consistent frequency of **3.975 GHz**, indicating that the processor was boosting effectively despite the heavy stalls.

2. Instruction Efficiency

- **Catastrophic IPC:** The system achieves a dismal aggregate IPC (Instructions Per Cycle) of **0.16**. This is exceptionally low, meaning the CPU is spending roughly 6 clock cycles for every single instruction retired.
- **Branch Prediction:** The branch predictor is performing very well, with a **0.65% branch-miss rate**. Out of over 14.4 billion branches, the CPU missed less than 1%, confirming that the control flow of the finite difference loops is highly regular and predictable.

3. Primary Bottlenecks

- **Memory Bound (The "Backend" Wall):** The defining characteristic of this simulation is a severe memory bottleneck.
 - **Frontend is Clear:** The stalled-cycles-frontend is only **1.44%**. The CPU is having no trouble fetching or decoding instructions.
 - **Backend Stalled:** Since the frontend is working perfectly but the IPC is 0.16, the execution units are effectively stalled for the vast majority of the runtime. The processors are starved for data, spending almost all their time waiting on RAM to fetch the 2D grid neighbors required for the stencil computation.
- **Context Switching:** The system is performing roughly **116 context switches per second**, which is relatively low, suggesting the bottleneck is not OS scheduling overhead but rather raw hardware bandwidth saturation.